# Mastering 2048 with AlphaZero

**Bob Qian**

## Introduction:

AlphaZero is a reinforcement learning algorithm that learn how to make decisions through interaction and feedback, rather than by copying human examples. The core idea behind both is to combine deep neural networks, which estimate what actions are promising and how good a situation is, with Monte Carlo Tree Search, a planning process that looks ahead and turns experience into better learning signals. This approach made AlphaZero a major breakthrough because it could learn to play chess, shogi, and Go at a superhuman level starting only from the rules and self-play, without hand designed strategies or human data [1]. AlphaZero matter beyond games because many real-world problems, such as robot control, scheduling, and resource allocation, also involve long term planning with delayed rewards and uncertain or hard to model dynamics.

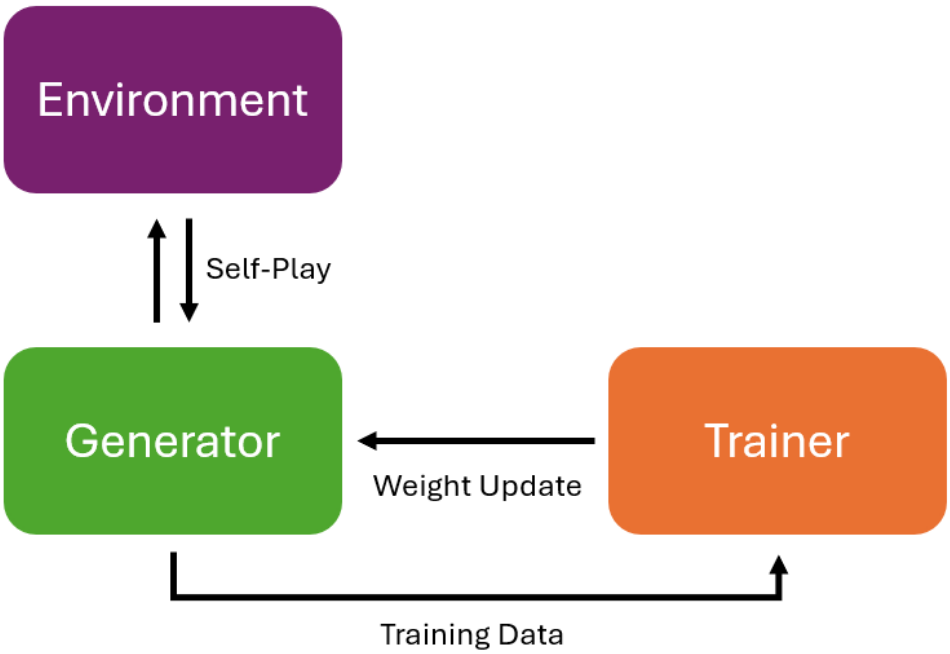


*Figure 1. Overall structure of the AlphaZero-style training pipeline. The Generator interacts with the Environment through self-play to produce training data, which is used by the Trainer to update the neural network parameters. The updated weights are fed back into the Generator, enabling iterative performance improvement through repeated self-play and training cycles.*

## Theoretical Explanation:

One of the most important components of AlphaZero is self-play. The quality of the data collected through self-play largely determines the strength of the agent. Self-play can be divided into two parts: Monte Carlo Tree Search (MCTS) and environment stepping. In each episode, the agent first uses MCTS to produce an improved policy over actions (e.g., via visit counts), then selects an action from this distribution (often by sampling with a temperature parameter, rather than always taking the argmax), executes that action, and steps the environment. This process is repeated until the episode ends.

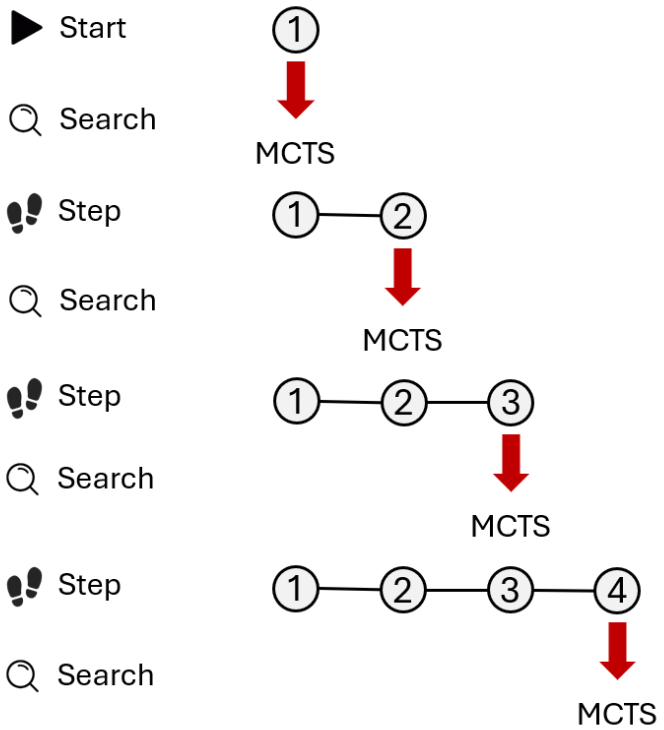


*Figure 2. AlphaZero Self-Play Loop: At each timestep, the agent alternates between Monte Carlo Tree Search (to select the action) and environment stepping (to execute it), progressively building an episode until termination.*

A typical Monte Carlo Tree Search (MCTS) can be divided into four stages: selection, expansion, simulation/evaluation, and backup. In selection, the search starts at the root and repeatedly chooses a child according to a tree policy (e.g., UCT/PUCT) that trades off exploiting high-value nodes and exploring less-visited ones, stopping when it reaches a terminal node or a node that is not fully expanded. In expansion, if the node is non-terminal and has untried actions, the algorithm adds a new child by applying one of those actions to create a new state in the tree. In simulation/evaluation, the algorithm estimates the value of that leaf state either by performing a rollout with a default policy or by directly using a value function/model to predict the expected outcome (often also

providing policy priors). In backup, the resulting value is propagated back along the selected path to the root, updating each node's statistics (such as visit counts and value estimates) to guide future selections.

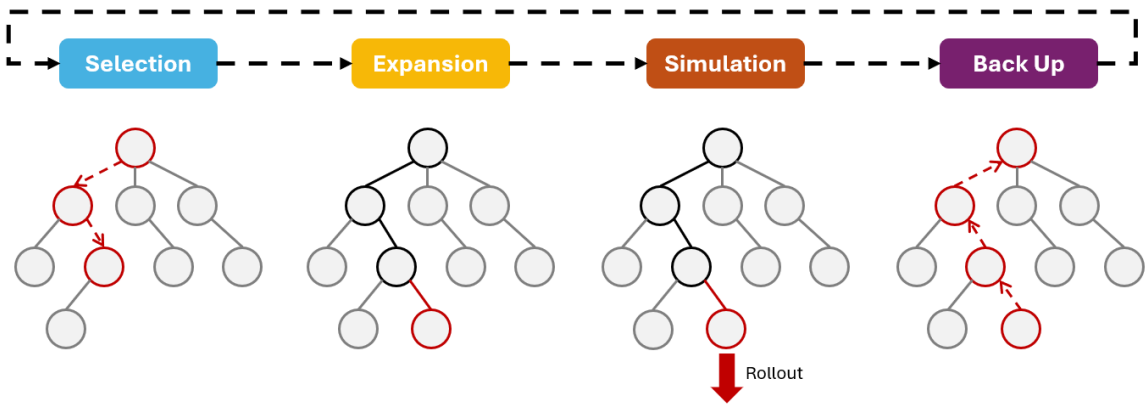


*Figure 3. One iteration of Monte Carlo Tree Search (MCTS). The algorithm selects a path from the root using a tree policy, expands the tree by adding a new child, simulates/evaluates the resulting state (e.g., via rollout), and then backs up the obtained value to update statistics along the visited path.*

AlphaZero uses PUCT instead of UCT in MCTS because it has a neural network that provides a policy prior $P(s,a)$ and a value estimate, so the search can be guided toward promising moves while still exploring alternatives. PUCT adds an exploration bonus weighted by the policy prior, focusing visits where the network predicts good actions and making MCTS far more sample-efficient in large branching-factor games than UCT, which explores mostly based on visit counts without informed priors.

$$\text{UCT}: \quad a^* = arg\max_a \left[ Q(s,a) + c\sqrt{\frac{\ln N(s)}{N(s,a)}} \right]$$

$$\text{PUCT}: \quad a^* = arg\max_a [Q(s,a) + U(s,a)]$$

$$U(s,a) = c_{puct} p(a|s) \frac{\sqrt{N(s)}}{1 + N(s,a)}$$

*Figure 4. UCT/PUCT choose the action $a^*$ at state $s$ that maximizes a score made of exploitation $Q(s,a)$ plus an exploration bonus: in UCT, the bonus is $c\sqrt{ln\ N(s)/N(s,a)}$, where $N(s)$ is how often the state was visited, $N(s,a)$ is how often that action was tried from the state, and ccontrols the explore–exploit tradeoff; in PUCT, the bonus is $U(s,a) = c_{puct}\, p(a \mid s)\, \frac{\sqrt{N(s)}}{1+N(s,a)}$, which plays the same role but additionally uses a prior $p(a \mid s)$ to bias exploration toward promising actions while still shrinking the bonus as $N(s,a)$grows.*

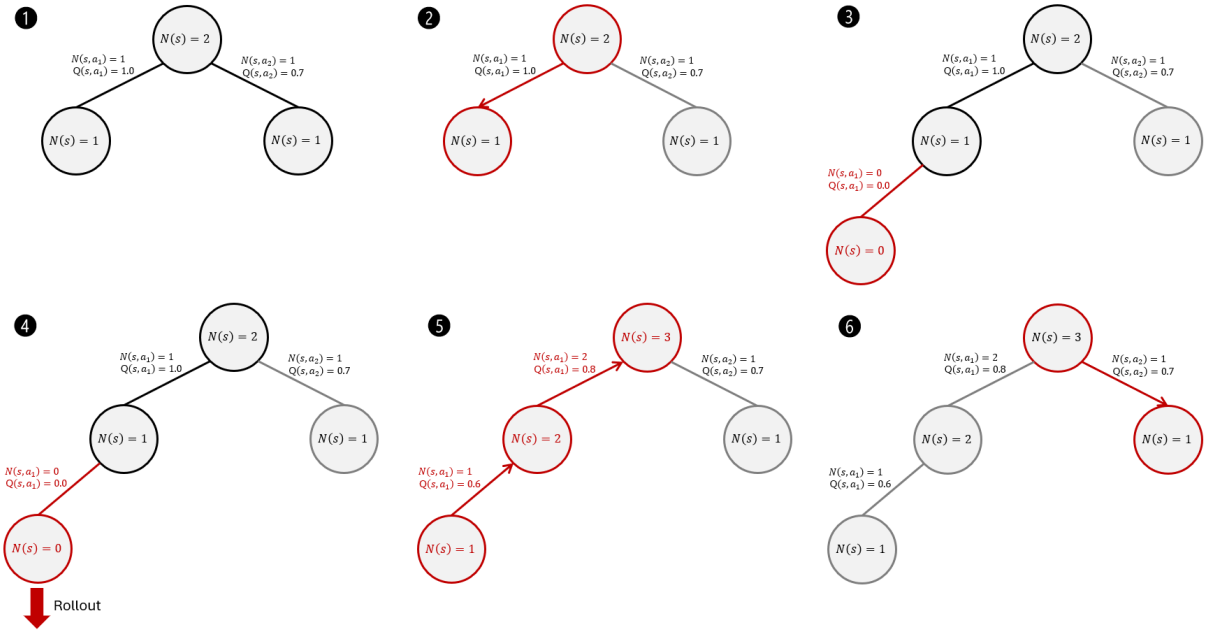


*Figure 5. UCT (Upper Confidence bounds applied to Trees) in MCTS, one full iteration.*
*(1) Start with visit counts $N(s)$and action stats $N(s,a), Q(s,a)$. (2) Selection: follow the child maximizing the UCT score (red path). (3) Expansion: add a new leaf (initially $N = 0, Q = 0$). (4) Rollout/Simulation: play out from the new leaf to get a return. (5) Backup: update $N$and $Q$along the visited path. (6) Next selection shifts toward less-visited actions, illustrating the exploration–exploitation tradeoff.*

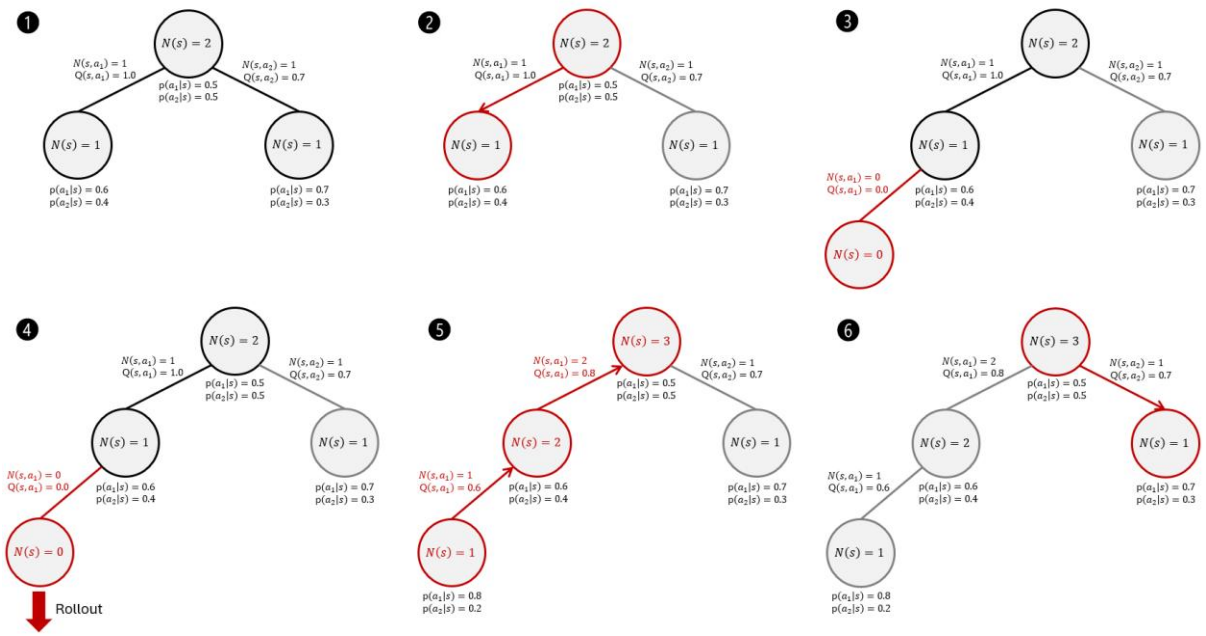


*Figure 6. PUCT (Policy + UCT) in MCTS, one full iteration with priors.*
*(1) Each edge stores $N(s,a), Q(s,a)$plus a policy prior $p(a \mid s)$from a model. (2) Selection: follow the action maximizing the PUCT score, where the exploration bonus is weighted by $p(a \mid s)$(red path). (3) Expansion: add a new leaf with its own priors and initialize $N = 0, Q = 0$. (4) Rollout/Value estimate: obtain a return (or network value) from the leaf. (5) Backpropagation: update $N$and $Q$along the selected path. (6) Subsequent selections increasingly balance high-value actions with high-prior actions, showing how priors guide exploration early while visits gradually dominate.*

After each episode finished, we push $(s_t, \pi_t, \gamma_t)$ into the replay buffer: the board state, the MCTS-improved policy from root visit counts over {up, down, left, right}, and a value target (typically the discounted return / normalized final score). The trainer samples mini-batches from the buffer and updates the network with CE loss so the policy head matches $\pi_t$ and the value head matches $\gamma_t$. The updated network is then used to provide priors and leaf values for the next round of MCTS self-play, repeating the loop.

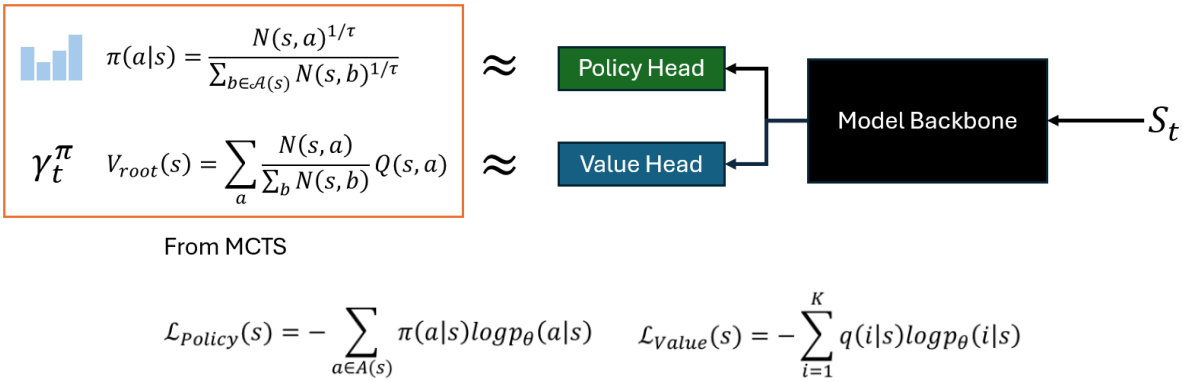


*Figure 7. MCTS-to-network training targets: from state $s_t$, MCTS produces an improved policy $\pi(a \mid s)$ (visit-count distribution) and a root value estimate $V_{root}(s)$ (visit-weighted $Q$) as loss target. These targets supervise the model's shared backbone via a policy head and value head using cross-entropy losses $\mathcal{L}_{Policy}$ and $\mathcal{L}_{Value}$*

# Code Demonstration:

To better understand AlphaZero, I decided to implement it from scratch with the help of GPT-5.1 Thinking. The full implementation is available at [bobqianic/AlphaZero-2048, an efficient C++ implementation of AlphaZero for 2048](bobqianic/AlphaZero-2048, an efficient C++ implementation of AlphaZero for 2048).

The initial version was written in PyTorch, but I quickly realized that naïve MCTS is inherently sequential, meaning most of the runtime is spent on node traversal while the GPU remains largely underutilized due to the lack of large batches. A common approach to address this is parallel PUCT with virtual loss, which allows batching evaluations of multiple leaf nodes, but at the cost of introducing bias and additional complexity.

Instead, I used GPT-5.1 to translate the PyTorch code into LibTorch C++ and implemented an epoll-like scheduler to enable asynchronous MCTS execution. This design significantly improved GPU utilization, increasing it from about 10% to 50% on single RTX 5090.

In 2048, the environment is stochastic, so we model it by inserting a chance node after each player move. The chance node represents the afterstate (the board after the move but before the tile spawn). When we expand a chance node, we randomly sample a valid spawn (position and tile value) to produce the next successor state. Therefore, when we back up the value to the root, we need to account for the chance step. Since

it's part of the same environment transition, we set $\gamma = 1$ when passing through chance nodes to avoid applying discount twice.

```cpp
static BackupValue move_to_parent(const BackupValue& v_child,
                                  const State& /*parent_state*/,
                                  const State& child_state) {
  const double r = child_state.reward_from_parent_raw;
  const double gamma = child_state.gamma_from_parent;
  return r + gamma * v_child;
}
```

*Figure 8. C++ function computing a backup value by backup a child state's value to its parent using reward and discount (γ).*

Another key point that people often overlook is that policy sampling and environment stepping must use two different random number generator streams. Otherwise, reward hacking can occur. Because most random number generators are pseudorandom, a fixed seed produces a fixed sequence. Since we always sample the policy and then step the environment, using the same RNG stream allows the model to bias the policy toward that specific sequence, causing tiles to spawn in more favorable positions. However, this behavior only holds for that particular seed. As training continues, the model overfits to it and fails to generalize when the RNG settings change.

```cpp
ctx->ep_seed = splitmix64(x: cfg_base.seed ^ (0xA5A5A5A5A5A5A5A5ull + (std::uint64_t)ep));
ctx->game_rng   = env2048::RNG(seed: splitmix64(x: ctx->ep_seed ^ 0x1111111111111111ull));
ctx->sample_rng = env2048::RNG(seed: splitmix64(x: ctx->ep_seed ^ 0x2222222222222222ull));
```

*Figure 9. Separating RNG streams for environment dynamics and policy sampling using splitmix64-derived seeds to prevent seed-specific reward hacking and improve generalization.*

```cpp
struct MinMaxStats {
  double minimum =  std::numeric_limits<double>::infinity();
  double maximum = -std::numeric_limits<double>::infinity();

  void reset() {
    minimum =  std::numeric_limits<double>::infinity();
    maximum = -std::numeric_limits<double>::infinity();
  }

  void update(double value) {
    if (!std::isfinite(value)) return;
    minimum = std::min(Left: minimum,  Right: value);
    maximum = std::max(Left: maximum,  Right: value);
  }

  double normalize(double value) const {
    if (!std::isfinite(value)) return 0.5;
    if (maximum > minimum + 1e-12) {
      return (value - minimum) / (maximum - minimum);
    }
    return 0.5;
  }
};
```

*Figure 10. C++ Min–Max utility struct that tracks numeric bounds, safely updates with finite values, and normalizes inputs with a stable fallback.*

In 2048, unlike board games such as Go, the value is not naturally bounded between −1 and 1. Instead, it is always positive and increases rapidly after merges. Therefore, we use MinMaxStats (Fig. 10) to normalize values within MCTS.

Similarly, the model cannot be trained on unbounded values, as this causes the gradients to become unstable and makes training difficult. To address this, we use a categorical representation combined with an invertible transformation

$h(x) = sign(x)(\sqrt{|x|+1} - 1 + \epsilon x)$ where $\epsilon = 0.001$, to scale the target values.

Min–max normalization only works inside MCTS not in training because it's applied locally to values that are only used for relative comparisons during that search, so the exact scale doesn't need to be consistent across time. During model training, however, the target value distribution shifts as the policy and search improve, so min/max statistics drift. The same true value would map to different normalized targets over time, adding non-stationarity and destabilizing learning.

```
float muz_h(float x, float eps) {
  const float s  = (x >= 0.0f) ? 1.0f : -1.0f;
  const float ax = std::abs(x);
  return s * (std::sqrt(ax + 1.0f) - 1.0f) + eps * x;
}

float muz_h_inv(float y, float eps) {
  const float s  = (y >= 0.0f) ? 1.0f : -1.0f;
  const float ay = std::abs(y);
  const float inside = 1.0f + 4.0f * eps * (ay + 1.0f + eps);
  const float t = (std::sqrt(inside) - 1.0f) / (2.0f * eps);
  const float x = t * t - 1.0f;
  return s * x;
}
```

*Figure 11. Value scaling and its inverse. C++ implementations of the transform $h(x)$and the corresponding inverse $h^{-1}(y)$using $\varepsilon$-regularized signed square-root scaling.*

```
torch::Tensor decode_value_raw(torch::Tensor value_logits, const torch::Tensor& support, float eps) {
  // value_logits: [B,601] on same device as support
  auto prob :Tensor = torch::softmax(value_logits, dim: 1);        // [B,601]
  auto expected_y :Tensor = (prob * support).sum( dim: 1);        // [B]
  return muz_h_inv_tensor(expected_y, eps);                        // [B], stays on device
}
```

*Figure 12. Decoding a scalar value from distributional logits. LibTorch code applies softmax over value logits, computes the expected value on the discrete support, then maps it back to the raw value space via the inverse transform.*

## Results and Visualisation:

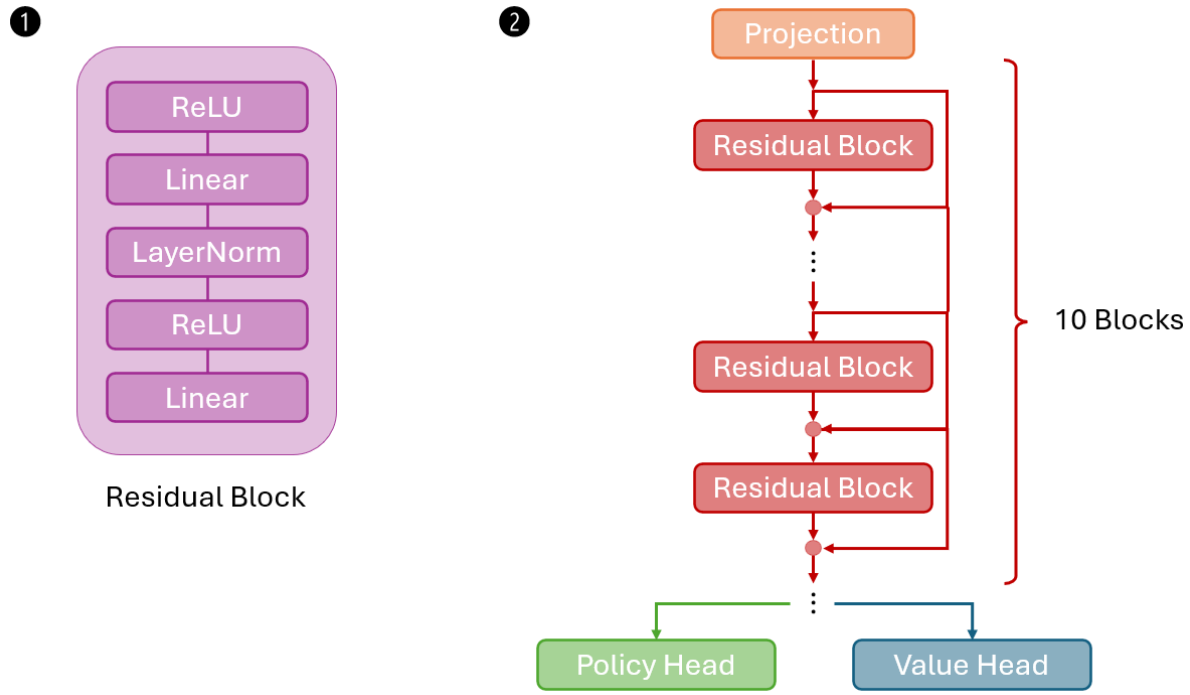


*Figure 13. Architecture of the model used in our experiments. Left (Residual block): The left panel shows the internal structure of one residual block. Right (Full model): The right panel shows the complete network used in our experiments: The input is first passed through a projection layer that projects the input into a latent space and then processed by 10 residual blocks stacked sequentially. After the final block, the shared features branch into two outputs: a policy head and a value head, and each head is a single Linear layer.*

Compared with supervised learning, reinforcement learning is often more sensitive to training "knobs" such as replay buffer size and refresh policy, and the balance between data collection (episodes per iteration) and optimization (gradient steps per iteration). Using the same replay buffer size, refresh policy, weight initialization, training hyperparameters and seed, we compare four schedules (e.g., 1ke-1kt = 1k episodes and 1k training steps per iteration). We observe that v2-1ke-1kt (blue) learns fastest early but saturates; v2-1ke-0.2kt (orange) improves more slowly at first yet achieves the best final reward; v2-5ke-1kt (purple) improves steadily and finishes close behind; and the afterstate variant (green) underperforms throughout.

Prior works on 2048 [2][3] commonly reports benefits from afterstate value learning (the state after the player's move and merges, before the random tile appears). However, we do not see a gain here in our AlphaZero-style setup. With a joint policy+value network, incorporating afterstates can increase the number of network evaluations inside MCTS, because policy is needed at player decision nodes, while value must be estimated at afterstate/chance transitions, which in our implementation doubled inference cost. Given the added compute and the lower reward curve, the benefit appears questionable in this setting.

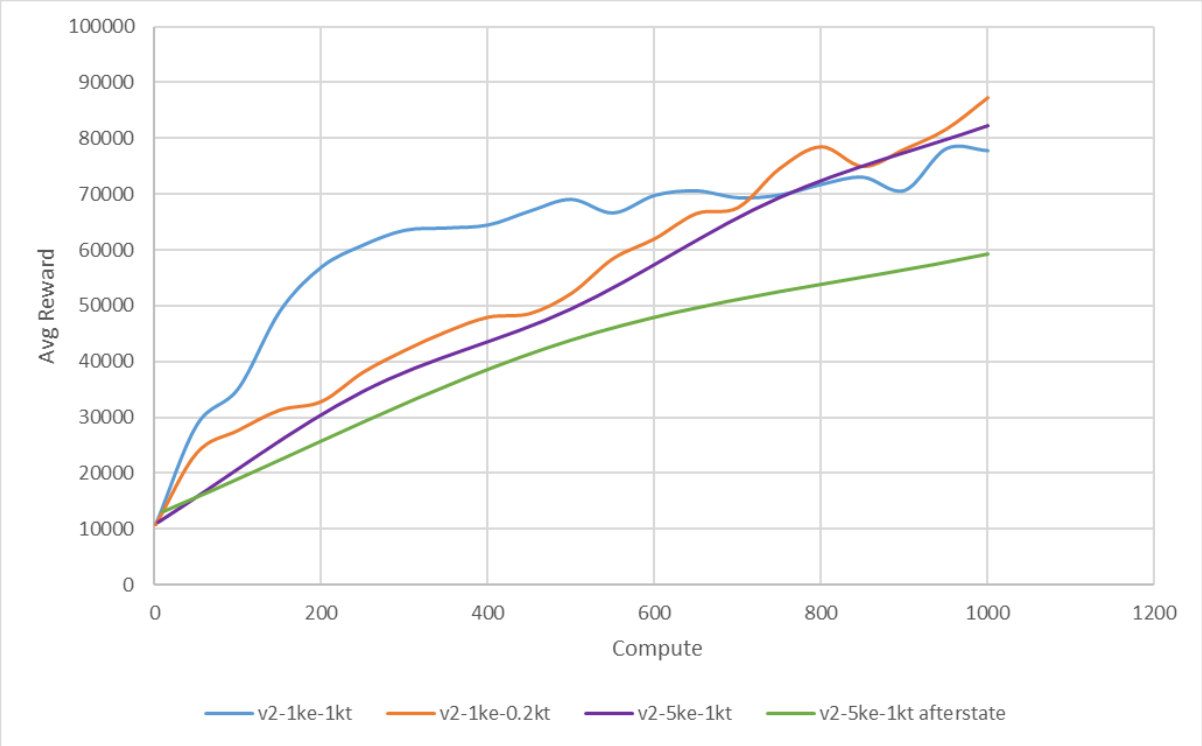


*Figure 14. Average reward (mean over 1,000 evaluation games) vs. compute for four AlphaZero training schedules, using the same weight initialization and seed. All curves are plotted starting from the end of the first training iteration, so curves do not start at 0 compute.*

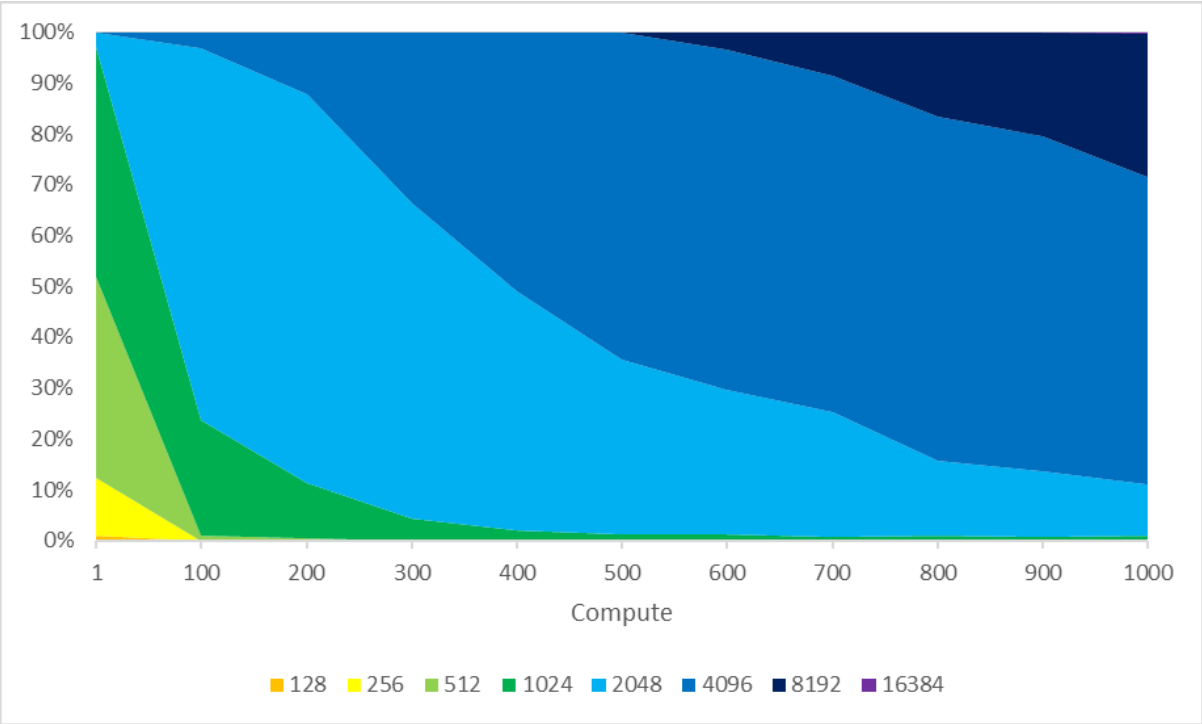


*Figure 15. Max-tile distribution versus compute for the best setup (v2-1ke-0.2kt). MCTS Sim=128.*

For the best setup v2-1ke-0.2kt, increasing compute steadily pushes the typical maximum tile upward. At compute 1, most games peak at 1024 (about 45%) or 512 (about 40%), with only a small share reaching 2048 (about 3%). By compute 100 to 200, 2048 becomes the clear ceiling for most runs (roughly mid-70%), while 4096 is still relatively uncommon (low double digits). Around compute 400 the distribution is essentially split between 2048 and 4096 (each about half), and from 600 through 900, 4096 dominates at roughly two-thirds of games. At the highest compute, gains show up mainly as more 8192 runs, rising from a few percent around 600 to about one-fifth by 900 and close to 30% at 1000, while 16384 remains almost never reached (around 0.1%).

```
Step 5649  action=Down  reward=8  score=133724  max_tile=8192
+------+------+------+------+
|     2|    16|   128|     2|
+------+------+------+------+
|     4|    64|   256|  2048|
+------+------+------+------+
|   128|  8192|    32|   512|
+------+------+------+------+
|     8|    16|  1024|     2|
+------+------+------+------+

=== Game Over ===
```

*Figure 16. Example run using my agent to play 2048, reaching an 8192 tile with a final score of 133,724.*

| | 128 | 256 | 512 | 1024 | 2048 | 4096 | 8192 | 16384 |
|---|---|---|---|---|---|---|---|---|
| sim=128 | 0.00% | 0.00% | 0.00% | 1.00% | 9.90% | 60.50% | 28.50% | 0.10% |
| sim=512 | 0.00% | 0.00% | 0.00% | 0.20% | 4.10% | 39.00% | 55.70% | 1.00% |

*Figure 17. Increasing the number of MCTS simulations at eval leads to a significant boost in performance.*

More MCTS simulations at eval consistently and significantly improves performance. This happens because, with a policy–value model, MCTS is not exploring blindly: the policy head provides strong priors that steer the search toward promising actions early, while the value head supplies informed leaf evaluations so the tree can estimate long-term returns without expensive rollouts. Increasing the number of simulations gives the search more chances to (1) correct mistakes from an imperfect policy prior, (2) reduce noise in value estimates by aggregating more visit statistics, and (3) concentrate visits on the highest-value branches via the selection rule, so the final action choice is based on more accurate Q/visit estimates.

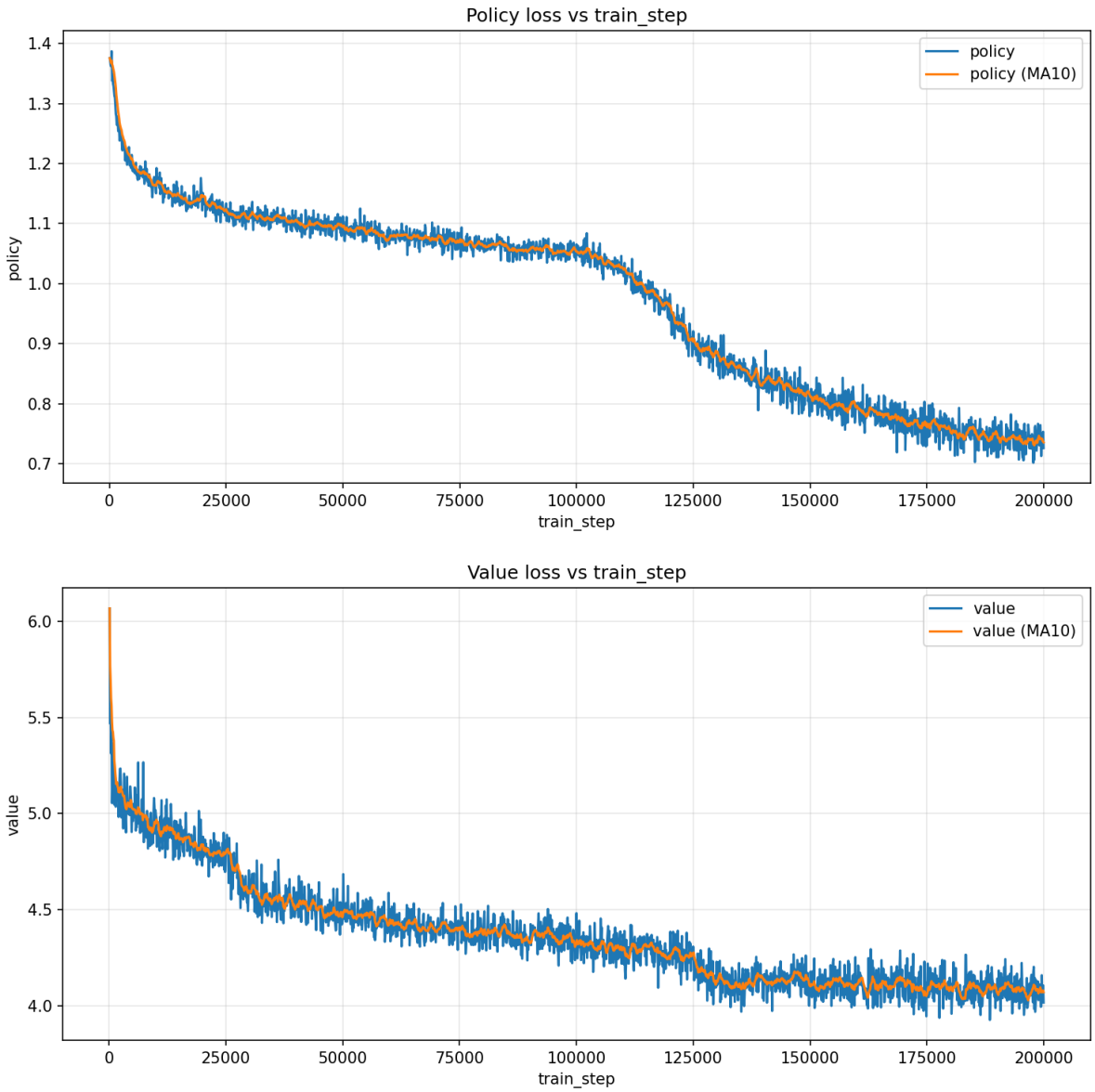


*Figure 18. Loss curves for the best setup (v2-1ke-0.2kt), using raw loss (no importance-sampling scaling)*

We use an Adam learning rate of 3e−4. Value targets are computed with n-step TD(λ) bootstrapping using n = 10, discount γ = 0.999, and λ = 0.5. Exploration is controlled via a temperature schedule [1.0, 0.5, 0.1] applied over the first [1e5, 2e5, 3e5] training steps, respectively. During self-play, we run MCTS with 100 simulations per move.

These training curves (best setup v2-1ke-0.2kt, raw unscaled losses) show stable, monotonic learning. The value loss steadily decreases, meaning the value head is getting better at predicting final outcomes as self-play improves. The policy loss also declines as the policy head better matches the MCTS target distribution, with a marked drop around 110k–130k steps, precisely when 8192 tiles first appear in self-play. That phase transition indicates the network has learned a qualitatively stronger strategy, leading to higher-quality search targets and faster loss reduction for both value and policy heads.

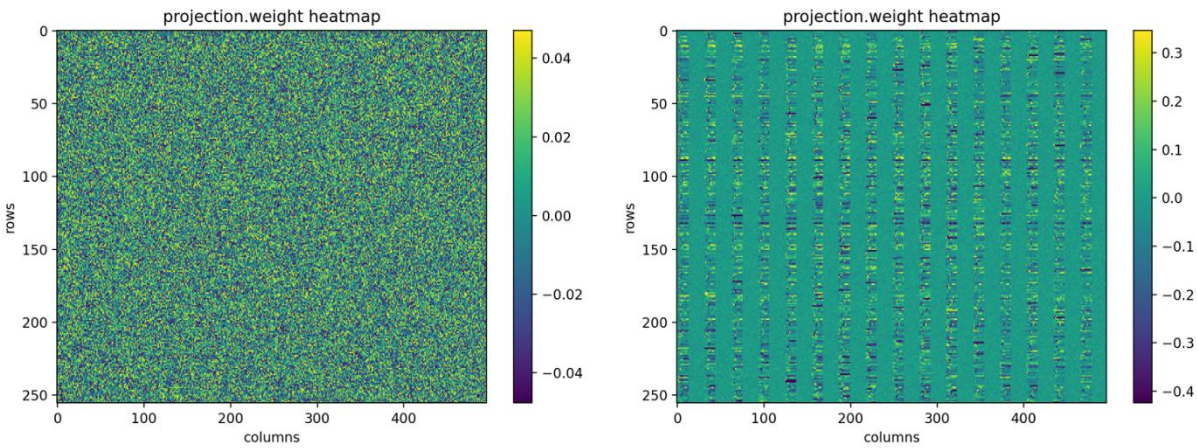


*Figure 19. Projection layer weight heatmaps across training. Left: weights after 1 iteration. Right: weights after 1000 iterations.*

Projection layer maps the 2048 board encoding into latent space. At initialization (left heatmap, after 1 iteration) the weights are essentially random noise with no visible structure. After training for 1,000 iterations (right heatmap), gradient updates have aligned the weights with meaningful patterns in the data: columns (input board cells/features) now show consistent vertical bands where many latent units respond similarly, while other columns are pushed closer to zero. This reflects the first layer learning which board positions or feature groups are informative for the task and how they should be combined, effectively reorganizing the raw board representation into a structured latent space that downstream layers can use more easily.

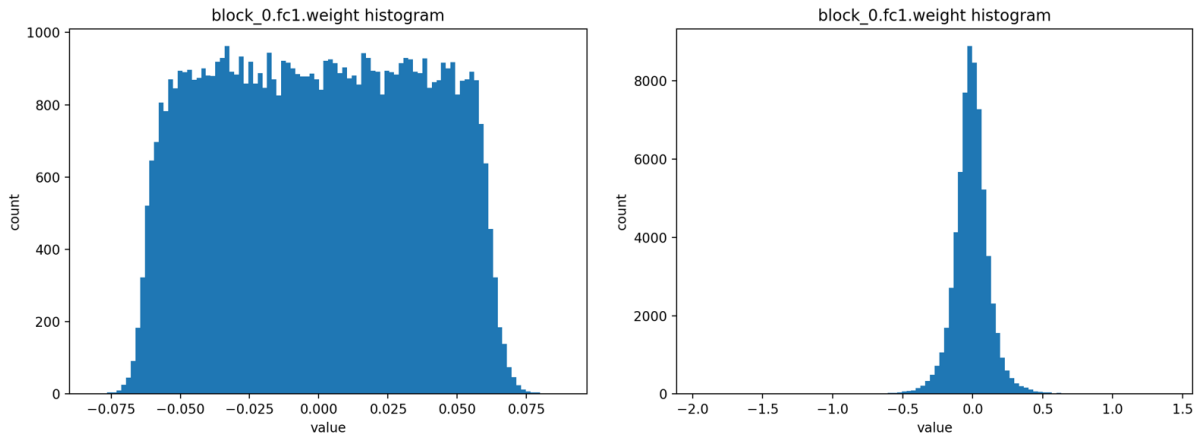


*Figure 20. Weight distribution shifts from near-uniform initialization after 1 iteration (left) to a bell-shaped, zero-centered distribution after 1000 iterations (right)*

This happens because each weight accumulates many small, noisy gradient updates, which shifts the initial uniform distribution toward a bell-shaped, zero-centered one. Weight decay (and optimizer bias toward smaller norms) pulls many weights closer to zero, while consistently useful connections grow larger, creating heavier tails, yielding lots of near-zero weights and a smaller set of strong, task-relevant weights.

## Reference:

[1]: https://arxiv.org/pdf/1712.01815

[2]: https://arxiv.org/pdf/2111.11090

[3]: https://www.jstage.jst.go.jp/article/ipsjjip/29/0/29_336/_pdf